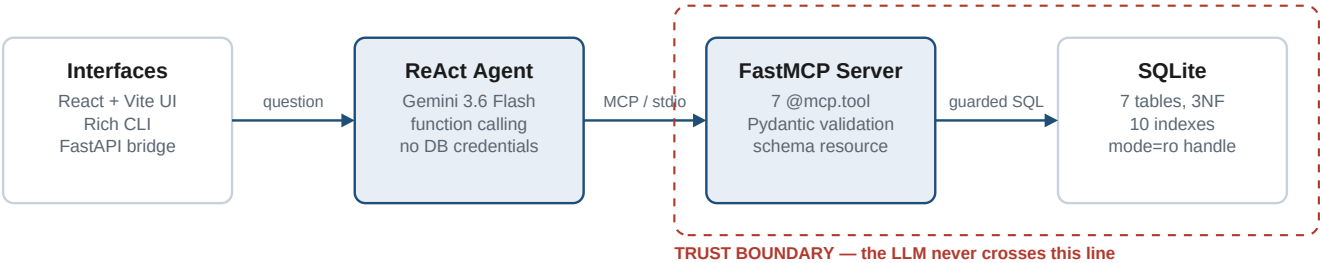


MCP-Enabled Database Query Agent

Natural-language querying over a relational database, with a hard read-only boundary



Pillar	Implementation
Agentic intelligence	ReAct loop (Reason → Act → Observe) over Gemini function calling, with dynamic tool routing and conversational schema memory
DBMS architecture	SQLite in 3NF: 7 tables, 10 indexes, 4,339 rows; three independent read-only enforcement layers; per-query plan and latency tracking
Protocol integration	FastMCP server exposing 7 tools, 1 resource and 1 prompt over stdio; Pydantic validates every tool argument at the protocol boundary

Python 3.13 · FastMCP 3.4 · Google Gemini 3.6 Flash · SQLite · FastAPI · React 18 + Vite · 85 passing tests

1. What the system does

A user asks a question in English — “*which department has the highest average grade points?*” An agent works out which tables hold the answer, writes the SQL, runs it, and replies with the number it actually retrieved.

The interesting part is not the translation. It is that the language model is structurally incapable of damaging the database. It has no connection string, no file path, and no credential. It can only name a tool and supply arguments, and every tool that reaches data passes through three independent read-only controls. A prompt injection that convinces the model to drop a table produces a rejection message, not a dropped table.

That separation is what makes the architecture worth building. The intelligence layer and the data layer are joined by a narrow, typed, auditable protocol instead of a database driver.

A worked example

you > Which department has the highest average grade points?

```
REASON  I need the schema before I can name any tables.
ACT      get_database_schema
OBSERVE  received database, note, relationships, tables          (12 ms)

REASON  Grades live in enrollments; department comes via students.
ACT      run_select_query
         SELECT d.dept_name, ROUND(AVG(e.grade_points), 2) AS avg_points
         FROM enrollments e
         JOIN students s      ON s.student_id = e.student_id
         JOIN departments d    ON d.dept_id    = s.dept_id
         GROUP BY d.dept_name
         ORDER BY avg_points DESC
OBSERVE  6 rows in 3.7 ms (index used)                          (18 ms)

ANSWER   Management Studies has the highest average at 7.14 grade points,
         narrowly ahead of Mathematics (7.06) and Computer Science (7.03).
```

Three details are worth pointing out. The agent fetched the schema before naming a single table — it is not permitted to guess. It derived a three-table join from the foreign-key edge list rather than from memorised conventions. And the result carries its own cost report: 3.7 ms, index used.

2. Architecture

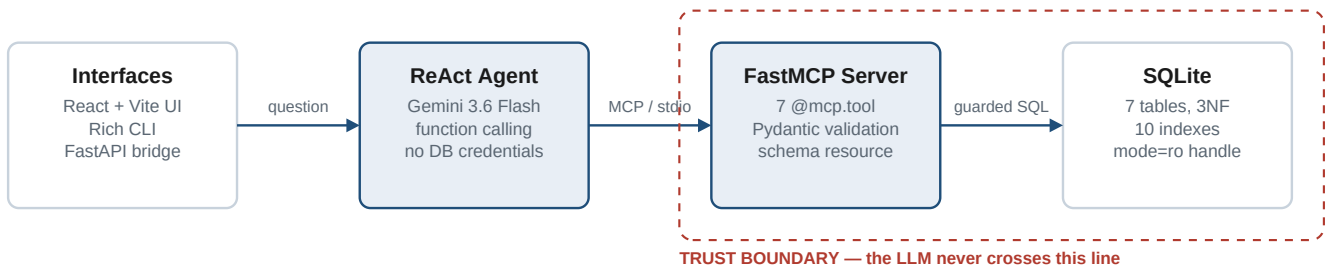


Figure 1 — Request path. Every arrow is a process or protocol boundary.

Four layers, each replaceable without touching the others:

Layer	Module	Responsibility
Interface	cli.py, api.py, frontend/	Collect a question, render the answer and the reasoning trace. Contains no database or model logic.
Agent	agent.py	Run the ReAct loop. Translates between Gemini's function-calling format and MCP tool calls. Holds conversation memory.
Protocol	server.py, mcp_client.py	Advertise tools with JSON Schemas, validate arguments, dispatch calls. The only crossing point between intelligence and data.
Data	database.py, guard.py	Own every SQLite connection. Enforce read-only access, cap rows, time queries, report query plans.

Why the boundary is real

The MCP server runs as a **separate operating-system process**. The agent launches it and speaks JSON-RPC over its stdin and stdout pipes. Nothing in the agent's address space can reach a database handle; there is no import path from `agent.py` to `database.py`. Moving the server to another machine would be a transport change, not a rewrite.

An in-process transport is also available and is what the test suite uses: the same MCP messages, the same validation, no subprocess. Switching between them is one environment variable, which is itself a demonstration of what the protocol abstraction buys.

Control flow of one question

- | | |
|----------------|---|
| 1. CLI / React | collects the question |
| 2. Agent | appends it to history, calls Gemini with 7 tool declarations |
| 3. Gemini | returns a thought + a function_call |
| 4. Agent | forwards the call over MCP as tools/call |
| 5. FastMCP | validates arguments against the tool's Pydantic model |
| 6. guard.py | parses the SQL, rejects anything that is not one SELECT |
| 7. database.py | opens mode=ro handle, installs authorizer, EXPLAINS, executes |
| 8. Agent | appends the result as a function_response, loops to step 3 |
| 9. Gemini | stops calling tools and answers in plain text |
| 10. Interface | renders the answer plus the full REASON/ACT/OBSERVE trace |

3. Technology choices

Each choice below was made against a specific alternative. The reasoning matters more than the selection.

Component	Chosen	Why, and what was rejected
Protocol	FastMCP 3.4	MCP is the emerging standard for connecting models to external systems — one integration works with any MCP-capable client. FastMCP reduces a tool to a decorated function and derives its JSON Schema from type hints. Writing a bespoke REST API would have meant hand-maintaining schemas that drift from the code.
LLM	Gemini 3.6 Flash	Free at AI Studio, native parallel function calling, and exposed reasoning traces (<code>include_thoughts</code>) that become the REASON lines of the visible trace. The agent code is provider-shaped only in one function; swapping providers means rewriting <code>_to_function_declaration</code> .
Database	SQLite	Zero-install, ships with Python, and — critically — supports both a read-only connection URI and a per-operation authorizer callback, which is what makes a genuinely layered security story possible. PostgreSQL would give real read-only roles but requires a running server.
SQL parsing	sqlglot	A real SQL parser produces an AST that can be walked for forbidden node types. Regular expressions over SQL are defeated by comments, string literals, and whitespace, and were rejected for that reason.
Validation	Pydantic 2	Used by FastMCP to enforce argument types and ranges before a tool body executes. A model that sends <code>max_rows=9999</code> against a 1–500 bound is refused at the protocol layer.
Backend	FastAPI	Async, so the single shared MCP session is not blocked by concurrent requests; automatic OpenAPI docs at <code>/docs</code> .
Frontend	React 18 + Vite	The UI's job is to make the ReAct trace inspectable, which is component state. No UI framework or state library was added — the app is small enough that hooks and plain CSS are the right size.
CLI	Typer + Rich	Coloured, structured trace output in a terminal, which is how the project is best demonstrated live.

4. Database design

Seven tables in third normal form. Normalisation is not decoration here: it is what makes the schema unambiguous enough for a model to navigate. Every fact is stored once, so there is exactly one correct join path to any answer, and the foreign-key graph fully describes it.

```

departments ---< professors      one department, many professors
departments ---< courses        one department, many courses
departments ---< students       one department, many students

course_offerings ---> courses    which course is being taught
course_offerings ---> professors who teaches it
course_offerings ---> semesters  when it runs

enrollments ---> students        who enrolled
enrollments ---> course_offerings in which offering (carries grade)

---< one-to-many      ---> foreign key reference

```

Table	Rows	Role
departments	6	Root entity
professors	34	FK → departments
students	420	FK → departments; cgpa, enrollment_year, status
courses	22	FK → departments; unique course_code
semesters	6	Unique (term, year)
course_offerings	84	Junction resolving course × professor × semester
enrollments	3,767	Junction resolving student × offering; carries grade and grade_points

The path `students → enrollments → course_offerings → courses → departments` is deliberate. Most interesting questions need three or four joins, so the agent cannot succeed by pattern-matching a single table — it has to read the relationship graph. That is the behaviour the schema was designed to force.

Indexing

SQLite indexes primary keys and unique constraints automatically, but **not** foreign keys. Without explicit indexes every join above would degrade to a nested full scan. Ten indexes cover all foreign keys plus two common filters, and ANALYZE runs at build time so the planner has statistics to cost them with.

Query	Plan	Latency
WHERE student_id = 42	SEARCH enrollments USING INDEX idx_enrollments_student	0.044 ms
WHERE first_name = 'Isha'	SCAN students (no index)	0.454 ms
3-table join, GROUP BY department	SEARCH via idx_enrollments_student + PK lookups	3.75 ms
4-table join, GROUP BY course	SEARCH via idx_offerings_course + PK lookups	4.44 ms

A ten-fold difference on 3,767 rows is small in absolute terms; the same ratio on a production table is the difference between a dashboard that loads and one that times out. The point of measuring it is that the agent is *told* the ratio and can act on it.

5. The read-only boundary

This is the part of the system that has to survive scrutiny. A model can be talked into a destructive query by text it reads inside a database row, and it can simply hallucinate one. The design assumption is therefore that **the model will eventually emit a DELETE**, and the system must be correct when it does. Three controls sit in series; each would stop a write on its own, and they fail in different ways, so a bug in one does not compromise the others.

#	Control	Mechanism	Defeated by
1	Static analysis <code>guard.py</code>	sqlglot parses the statement. The root must be SELECT; no INSERT, UPDATE, DELETE, DROP, ALTER, ATTACH, PRAGMA or transaction node may appear anywhere in the tree; exactly one statement is allowed.	A parser bug, or a construct sqlglot normalises away
2	Engine authorizer <code>database.py</code>	sqlite3.set_authorizer() is consulted by the engine for every operation a prepared statement attempts. Only SELECT, READ, FUNCTION and RECURSIVE are allowed; it is an allowlist, so unknown verbs fail closed.	Nothing in-process — this runs inside SQLite
3	Read-only handle <code>database.py</code>	The connection is opened as <code>file:university.db?mode=ro</code> . The driver itself refuses any write before SQL is even considered.	Filesystem-level access outside the process

Layer 1 exists mainly to produce a *useful error message* the agent can read and recover from. Layers 2 and 3 are what make a write impossible. Stating that distinction is the honest version of the defence: the friendly layer is not the one doing the security work.

Supporting controls

- **Stacked-statement rejection.** `SELECT 1; DROP TABLE students` is refused by statement count — the classic injection shape.
- **Row cap.** Results are limited (default 200) so one query cannot flood the model's context window; a probe row beyond the cap detects and reports truncation.
- **Statement timeout.** SQLite's progress handler interrupts any query running beyond 5 seconds, bounding an accidental cartesian join.
- **Internal tables blocked.** `sqlite_master` is denied by the authorizer; schema reaches the model only through the curated `describe_table` output.
- **Filesystem functions blocked.** `load_extension`, `readfile` and `writefile` are rejected by name.
- **Length limit.** Statements over 8 KB are refused before parsing.

How it is proved

`tests/test_guard.py` fires 18 write and injection payloads at layer 1. More importantly, `tests/test_database.py` tests layers 2 and 3 *independently*: it calls `sqlite3.execute("DELETE FROM students")` on a raw connection, bypassing the guard completely, and asserts the write still fails. A security layer that is only ever tested through the layer in front of it has not been tested.

```
$ dbagent query "SELECT 1; DROP TABLE students"
Rejected by read-only guard: Only one statement per call is allowed (2 were supplied).
```

```
$ dbagent query "ATTACH DATABASE '/etc/passwd' AS leak"
Rejected by read-only guard: ATTACH is not permitted
```

6. The ReAct agent

ReAct interleaves reasoning with tool use instead of separating them. The model does not plan the whole query up front; it takes one step, sees the real result, and decides again. That is what lets it recover from its own mistakes.

Phase	What happens
REASON	Gemini produces a thought about what it still needs to know. With <code>include_thoughts=True</code> this is the model's genuine reasoning, not a paraphrase.
ACT	It emits a <code>function_call</code> . The agent forwards it over MCP without interpreting it.
OBSERVE	The tool result is appended to the conversation as a <code>function_response</code> and the loop repeats, bounded at 10 iterations.

Dynamic tool routing

Tool declarations are generated at runtime from whatever the MCP server advertises. MCP publishes plain JSON Schema and the Gemini SDK accepts it verbatim through `parameters_json_schema`, so no field-by-field mapping exists that could drift:

```
def _to_function_declaration(spec: ToolSpec) -> types.FunctionDeclaration:
    schema = dict(spec.input_schema)
    schema.pop("title", None)
    return types.FunctionDeclaration(
        name=spec.name,
        description=spec.description[:1024],
        parameters_json_schema=schema,      # MCP JSON Schema, passed straight through
    )
```

The consequence: adding an `@mcp.tool` function to `server.py` makes it available to the model on the next start, with no change to `agent.py`. The interface is decoupled from the backend execution logic — which is the property the protocol exists to provide.

Contextual memory

Conversation history persists across turns on the agent instance, so the schema fetched for the first question is still in context for the fifth. Two things follow. Follow-ups work — *“now break that down by year”* resolves against the previous query. And the agent stops re-fetching the schema, because it already knows the join graph. Large observations are truncated before re-entering the context so a wide result set cannot crowd out the schema.

Failure handling

Tool errors are returned as data, never raised. A rejected write, a bad column name, or a timeout all arrive as an `error` field the model reads on its next turn — so it apologises and explains the read-only policy, or fixes the column name and retries. Raising would end the session; returning lets the loop do what it is for.

```
async def test_blocked_write_is_observed_not_raised(toolbox, scripted):
    scripted([
        _call("run_select_query", sql="DROP TABLE students"),
        _answer("That is not permitted; the database is read-only."),
    ])
    result = await DatabaseAgent(toolbox).ask("Delete all the students")

    observation = next(s for s in result.steps if s.kind == "observe").result
    assert observation["blocked"] is True
    assert "read-only guard" in observation["error"]
```

7. Operating inside a free-tier quota

This section exists because the naive implementation did not survive contact with the real API, and the failure was instructive.

Gemini's free tier enforces quotas **per model and per project** — a second API key in the same project shares the same budget. A single ReAct question spends three to six requests, one per loop iteration, so the daily column below is the binding constraint. An early test of five concurrent sessions exhausted the quota without a single one completing.

Model family	Req/min	Tokens/min	Req/day	Questions/day	Reasoning trace
Flash — 3.5 / 3.6 / 3.7	5	250K	20	~4	yes
Flash-Lite — 3.1 / 3.5	15	250K	500	~100	no

The last column is the design tension. Only Flash models emit the thought summaries that become the REASON lines of the trace; Lite models answer just as correctly but show only ACT and OBSERVE. The fallback order therefore puts Flash first — the full trace while the small budget lasts — and Lite behind it, so a demonstration degrades in fidelity rather than stopping.

Four mechanisms address this, and they are deliberately different in kind: one avoids the limit, one recovers from it, and two route around it.

Mechanism	Applies to	How it works
Client-side pacing <code>ratelimit.py</code>	Prevention	A sliding-window limiter admits at most N requests per 60 seconds, holding the rest. It is shared per model across the process, so every agent in the API server draws on one budget instead of each assuming it owns the quota. A sliding rather than fixed window is used because a fixed one permits a double-rate burst across its boundary — exactly the pattern that trips Google's limiter.
Retry with RetryInfo <code>agent.py</code>	Per-minute 429	Reads the <code>retryDelay</code> Google returns rather than guessing, falling back to exponential backoff when absent. Covers the case where another process shares the key.
Model fallback <code>agent.py</code>	Per-day 429	Switches to the next model in <code>GEMINI_FALLBACK_MODELS</code> . Each model has a separate daily allowance, so an exhausted one is stepped over rather than ending the session.
Model fallback <code>agent.py</code>	404 Not Found	Google retires models for new keys — <code>gemini-2.5-flash</code> now 404s for recently created projects. The old name never recovers, so this switches model instead of retrying.

The distinction that matters

A per-day exhaustion and a per-minute throttle are both HTTP 429, both carry `RESOURCE_EXHAUSTED`, and both suggest a `retryDelay` of about thirty seconds. Treating them alike is wrong in an expensive way: a daily cap will not clear for hours, so retrying wastes the user's time and, because rejected requests still count against the quota, can eat into the next day's allowance. The two are told apart only by the `quotald` inside the `QuotaFailure` detail block.

```
def daily_quota_exhausted(exc):
    for entry in _error_details(exc):
        if "QuotaFailure" not in (entry.get("@type") or ""):
            continue
        for violation in entry.get("violations") or []:
            if "PerDay" in (violation.get("quotaId") or ""):
                return {"limit": violation.get("quotaValue"), ...}
    return None      # per-minute: safe to wait and retry
```

Only when this returns a result does the agent stop retrying and change model.

Observed behaviour

One real question, exercising all four mechanisms in sequence — paced under the per-minute limit, stepped over an exhausted model, and retried a transient server error — before answering correctly:

```
QUEUE    pacing gemini-3.6-flash to 5 req/min - next slot in 18s
SWITCH   gemini-3.6-flash out of daily quota, switching to gemini-3.7-flash
RETRY    HTTP 503 (attempt 1) in 2s
ACT       get_database_schema
OBSERVE  received database, note, relationships, tables      (89 ms)
ACT       run_select_query
OBSERVE  2 rows in 0.6 ms (index)                            (55 ms)
```

For a system that has to work during a live demonstration, this is the difference between a working project and one that fails in front of an audience for reasons unrelated to its design.

8. Protocol integration

MCP standardises how a model reaches an external system — the same integration serves any MCP-capable client rather than being written once per model vendor. The server exposes all three MCP primitives.

Primitive	Name	Purpose
Tool	get_database_schema	Full schema plus the foreign-key edge list. The agent's first call.
Tool	list_tables	Table names with row counts.
Tool	describe_table	Columns, keys, indexes and DDL for one table.
Tool	sample_table_rows	A few real rows, so the model sees that status is 'active', not 'Active' — a common source of silently-empty WHERE clauses.
Tool	run_select_query	Guarded execution returning rows, timing and index usage.
Tool	explain_query_plan	Query plan without executing.
Tool	get_query_log	Session history with timings.
Resource	schema://university	The schema as attachable context for clients that prefer resources.
Prompt	analyse_question	A reusable template that walks any client through the ReAct loop.

What FastMCP removes

A tool is a decorated function. Its JSON Schema comes from the type hints and its description from the docstring, so the contract the model sees and the code that runs cannot disagree:

```
@mcp.tool
def run_select_query(
    sql: Annotated[str, Field(description="A single SELECT statement.")],
    max_rows: Annotated[int, Field(ge=1, le=500, description="Row cap (1-500)")] = 200,
) -> dict:
    """Execute one read-only SELECT and return the rows.

    Rejects anything that is not a single SELECT. On rejection, an `error` field
    explains why - read it and correct the SQL rather than retrying unchanged.
    """
```

The docstring is not documentation for a human reader; it is the instruction the model acts on. Telling the model *what to do when a call fails* inside the tool description measurably reduces retry loops, which is why these docstrings read the way they do.

Pydantic enforces the bounds before the body runs. A model that sends `max_rows=9999` is refused at the protocol boundary with a schema error, not by defensive code inside the function.

Transports

Transport	Shape	Used for
stdio	Server launched as a subprocess; JSON-RPC over pipes	Default. The real deployment shape — the server could equally be remote.
in-memory	Client wired to the server object; identical MCP messages	Tests. 85 tests run in ~4 s with no process spawning.

9. How it was built

The order mattered. Each stage was verified before the next depended on it, so no failure could be ambiguous about its origin.

Stage	Work	Verified by
1. Schema	Model the domain in 3NF, add CHECK constraints and foreign keys, index every FK, ANALYZE for planner statistics.	sqlite3 accepted the DDL; PRAGMA foreign_key_list returned the expected edges
2. Data	Deterministic generator seeded with a fixed value, with realistic correlations (the 2020 intake has mostly graduated; students take ~70% of courses in their own department).	Rebuilds byte-identically, so every figure quoted in this report reproduces
3. Guard	Allowlist SQL validation on a sqlglot AST.	18 attack payloads rejected, 6 legitimate query shapes accepted
4. Data layer	Read-only URI, authorizer callback, progress-handler timeout, EXPLAIN QUERY PLAN parsing, query log.	Raw sqlite3 writes bypassing the guard still fail
5. MCP server	Seven @mcp.tool functions, one resource, one prompt.	A FastMCP client listed the tools and executed a 3-table join over the protocol
6. Agent	ReAct loop; MCP schemas translated to Gemini function declarations; trace capture; conversation memory.	Loop mechanics tested against a scripted model — no API key, real MCP and SQLite
7. Interfaces	Rich CLI, FastAPI bridge with a shared MCP session, React UI with an expandable trace view.	End-to-end: browser → Vite proxy → FastAPI → MCP → SQLite

Two things that went wrong

Truncation was silently impossible to detect. The row cap was implemented by appending `LIMIT max_rows` and then checking whether more than `max_rows` rows came back — which it never could, because SQL had already truncated them. The flag read `false` on every oversized result. A test caught it; the fix is to request one row beyond the cap as a probe and trim before returning. Worth noting because the bug was invisible from the outside: the data was correct, only the warning was missing.

The MCP server's log output corrupted the CLI. Under stdio transport the server's startup banner was written to the same stream the client renders, interleaving ASCII art with query results. Fixed by disabling the banner and raising the subprocess log level through the transport's environment — a reminder that with stdio transports, `stdout` belongs to the protocol.

10. Testing and results

Suite	Tests	What it establishes
test_guard.py	29	18 write and injection payloads rejected; 6 legitimate shapes (CTE, UNION, correlated subquery, multi-join) accepted; row-limit injection correct
test_database.py	13	Authorizer and read-only handle block writes independently of the guard; index detection; FK introspection; row cap; query log
test_mcp_server.py	16	Every tool exercised over the real protocol; schemas well-formed; errors returned as data; Pydantic bounds enforced
test_agent.py	19	ReAct loop, tool dispatch, trace construction, conversation memory, step limit, retry/backoff, per-day vs per-minute quota, model fallback — against a scripted model, with real MCP and SQLite underneath
test_ratelimit.py	8	Sliding-window pacing on a fake clock: window arithmetic, concurrent callers never oversubscribing a window, per-model limiter sharing

85 tests, ~4 seconds. The suite needs no API key and no network, because the only non-deterministic component — the model — is the one part that is stubbed. Everything below it is real.

Measured performance

Operation	Latency
Indexed foreign-key lookup	0.04 ms
Full scan of students (420 rows, unindexed column)	0.45 ms
Three-table join with GROUP BY	3.75 ms
Four-table join with GROUP BY and HAVING	4.44 ms
MCP tool round trip, in-memory transport	< 1 ms
MCP tool round trip, stdio subprocess	~10–20 ms
Full agent turn (3 tool calls, Gemini 3.6 Flash)	2–5 s, dominated by model latency
Full agent turn when paced against the free-tier quota	up to 60 s of deliberate waiting

The model dominates end-to-end latency by three orders of magnitude. That is the argument for aggregating in SQL rather than pulling rows and computing in the loop: each avoided round trip saves seconds, while the query itself costs milliseconds.

Honest limitations

- **The model is not verified.** Guards prove the SQL is a read; nothing proves it answers the question asked. A semantically wrong but syntactically valid query returns a confident wrong answer. Mitigations used here — schema grounding, sample rows, forbidding invented numbers — reduce this but do not eliminate it.
- **Single-database scope.** The schema is fetched whole into context. A database with hundreds of tables would need retrieval over the schema rather than wholesale inclusion.
- **Server-side sessions.** Conversation state lives in process memory keyed by session id; it does not survive a restart and would not work across replicas without a shared store.

- **No authentication.** Every caller gets the same read-only view. Per-user row filtering would belong in the MCP server, not the agent — the agent must never be the component deciding what a user may see.
- **Free-tier throughput.** Roughly four questions per model per day (section 7). Pacing and model fallback make this survivable, not generous; sustained use needs a paid key.
- **No result caching.** The schema is re-fetched every session, costing one request out of a small daily budget that caching would recover.

11. Learning resources

Grouped by the concept each pillar depends on, with a note on what to take from each rather than a bare link.

Model Context Protocol

Resource	Take from it
Official specification — modelcontextprotocol.io	The three primitives (tools, resources, prompts) and the lifecycle. Read the architecture page before any tutorial.
FastMCP documentation — gofastmcp.com	Decorator API, transports, and the in-memory client used for testing here.
MCP Python SDK — github.com/modelcontextprotocol/python-sdk	The layer FastMCP sits on. Worth reading once to see the raw JSON-RPC messages behind the abstraction.
Anthropic's MCP introduction (Nov 2024)	The design rationale — why a protocol rather than per-vendor plugins.

Agentic patterns

Resource	Take from it
<i>ReAct: Synergizing Reasoning and Acting in Language Models</i> , Yao et al., arXiv:2210.03629	The original paper. Short and readable; the interleaving argument is the whole idea.
Gemini function-calling guide — ai.google.dev	Declaration format, parallel calls, and the manual loop this project uses instead of automatic function calling.
<i>Building Effective Agents</i> — Anthropic engineering blog	When a loop is warranted and when a fixed pipeline is better. A useful corrective to over-engineering.
<i>Prompt Engineering Guide</i> — promptingguide.ai	The ReAct and tool-use chapters, for how tool descriptions shape behaviour.

Database and security

Resource	Take from it
SQLite <i>Query Planner</i> and <i>EXPLAIN QUERY PLAN</i> docs	How to read SEARCH versus SCAN — directly what this project's used_index flag is built on.
<i>Use of sqlite3_set_authorizer()</i> — sqlite.org	The exact mechanism behind security layer 2.
<i>Database System Concepts</i> , Silberschatz, Korth, Sudarshan	Chapters 7–8 for normalisation, 14 for indexing. The standard reference if a normal-form question comes up.
<i>Use The Index, Luke</i> — use-the-index-luke.com	Practical, engine-agnostic indexing. The clearest explanation of why unindexed foreign keys are so costly.
OWASP <i>SQL Injection Prevention Cheat Sheet</i>	Allowlist-over-denylist reasoning, which is the principle both the guard and the authorizer follow.
OWASP <i>Top 10 for LLM Applications</i>	LLM01 Prompt Injection and LLM08 Excessive Agency name precisely the threats this architecture is built against.

Supporting stack

Resource	Take from it
sqlglot — github.com/tobymao/sqlglot	AST structure and the expression classes the guard walks.
FastAPI docs — lifespan events, dependency injection	The lifespan pattern used to share one MCP session across requests.
React docs (react.dev) — <i>Thinking in React</i>	State placement, which is the only real design question in this UI.
Pydantic v2 docs — validation and JSON Schema generation	What FastMCP does with type hints under the hood.

12. Anticipated questions

The questions this design most invites, with the answers it supports.

Question	Answer
<i>What stops the LLM dropping a table?</i>	Three independent controls, and the model never holds a database handle in the first place. Layer 1 gives a readable rejection; layers 2 and 3 make the write impossible. Layers 2 and 3 are tested by bypassing layer 1 entirely — a security layer only ever tested through the one in front of it is untested.
<i>Why MCP instead of a REST API?</i>	Schemas are generated from the code, so the model's view and the implementation cannot drift. Any MCP-capable client can use the same server. And the transport is swappable — this project runs the identical server over a subprocess in production and in-process in tests.
<i>Why not let the model connect directly?</i>	Then the credential is in the model's context, every guard becomes advice rather than enforcement, and there is no audit point. The narrow tool interface is what makes the boundary enforceable.
<i>How do you know the SQL is correct?</i>	Correctness of intent is not proven, and that limitation is stated plainly. What is enforced: the schema is read before any table is named, join paths come from the declared foreign-key graph, sample rows show real value formats, and the model is forbidden from stating a number no tool returned. Every trace is inspectable, so a wrong answer is diagnosable.

Question	Answer
<i>Why SQLite for a 'production-grade' system?</i>	Because it supports both a read-only handle and a per-operation authorizer, which is what makes the layered defence demonstrable on any machine. The data layer is one module; PostgreSQL would change that module and gain real read-only roles and EXPLAIN ANALYZE.
<i>What happens if the model loops?</i>	Bounded at 10 iterations, after which the loop returns an explanatory message. Each query is separately bounded by a 5-second timeout and a row cap. Tested.
<i>How would you scale this?</i>	Move the MCP server behind HTTP transport and run several agent workers against it; move session state to Redis; add per-user row filtering inside the MCP server, never in the agent. Cache schema responses — they change rarely and are fetched on every session.

13. Possible extensions

- **Result caching** keyed by normalised SQL, since schema and aggregate queries repeat heavily across a session — and, given the per-day request quota, caching the schema call alone would save one request per question.
- **Chart generation** — a tool returning a spec rather than an image, so the React layer renders it and the model never handles binary data.
- **Query approval mode** where the proposed SQL is shown for confirmation before execution — the standard pattern for higher-stakes deployments.
- **PostgreSQL adapter** behind the existing data-layer interface, adding real read-only roles and EXPLAIN ANALYZE.
- **Semantic evaluation set** — question/expected-answer pairs run in CI, which is the only real way to measure the correctness gap named in section 10.
- **Multi-database routing**, where the agent first selects a database and then a table, needed once the schema exceeds what fits in context.

Source, setup instructions and troubleshooting: README.md in the project root. Regenerate this report with `python scripts/build_report.py`.